BIG DATA · Academic year 2026-2027

Assignment 2

Optimised Dense and Sparse Matrix Multiplication in C

Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria

Carlos Falcón Brito

Repository: https://github.com/Cafabri/Assigment2

Abstract

The triple-loop `ijk` multiplication of Assignment 1 was compared in C with three optimised dense kernels (loop reordering `ikj`, cache blocking and a transposed operand) and with a CSR sparse multiplication (Gustavson's algorithm), on the performance cores of an Intel Core i7-13700KF (Windows 11), under a budget of 15 minutes per run. Every kernel adds the products of each element in the same order, and all results were bit-identical to `ijk` and agreed with NumPy and SciPy. For dense matrices, `ikj` needed 1.6–8.6 times less time than `ijk` for $500 \le n \le 4000$ and the blocked kernel 1.7–16.4 times less; the blocked kernel kept a constant cost of 0.22 ns per multiply-add from $n = 500$ to 4000, without the jump at $n = 2000$ seen in Assignment 1. Compiler vectorisation (AVX2) made the blocked kernel a further 2.5–2.8 times faster, up to 45.8 times faster than `ijk` at $n = 4000$, and changed the time of the kernel it could not vectorise by at most 4% from $n = 200$. At $n = 2000$, the CSR multiplication was faster than the scalar blocked kernel up to about 55% density for uniformly placed nonzeros and 57% for band matrices, and faster than the vectorised one up to 33% and 35%, with 10 100 and 18 900 times less time at 0.1% density. The CSR matrices needed less storage than the dense arrays up to 50% (uniform) and 56% (band) density, even when the product was almost full. CSR completed every tested size up to $n = 32\,000$, where uniform inputs with 1% density gave a product with 96% nonzeros and 11.2 GiB of storage. These results describe these programs on this computer.

1 Introduction and objective

1.1 Motivation

In Assignment 1 the naive `ijk` multiplication in C followed the n^3 model only up to $n = 1500$: between $n = 1500$ and 2000 its cost per multiply-add rose from 0.55 to 1.30 ns, which was attributed, without a direct measurement, to the column-by-column reading of B once B no longer fits in the L3 cache or in the reach of the TLB. If that explanation is right, changing only the order in which memory is read should remove most of the jump. A second way to save time is not to do the work at all: when most entries are zero, a sparse format stores and multiplies only the nonzeros, but the indexing it needs and the fill-in of the product can cancel that advantage [5].

1.2 Research questions

(A) How do loop reordering, cache blocking and a transposed operand change the runtime and memory of the ***`ijk`*** *dense multiplication of Assignment 1, and how much of the change comes from compiler vectorisation?*
(B) Under which tested densities and nonzero distributions does a CSR sparse multiplication need less time or memory than a dense method on the same logical matrices?

1.3 Variables

- **Factors varied:** Part A: the dense method, the matrix size n, the block size b and the build (scalar or AVX2). Part B: the density d of the inputs (the percentage of nonzero entries; the sparsity is $100\% - d$), the distribution of the nonzeros (uniform or band), n and the method (sparse or dense).
- **Outcomes measured:** the execution time of the kernel, the separately timed conversion and transposition costs, the nonzeros of the product and the memory.
- **Kept fixed:** the language (C), the compiler and its flags within each build, numeric type (float64), input values, machine, single thread.

1.4 Hypotheses

Table 1 lists the predictions, derived from Section 2. The predictions on the transposed operand and on the sparse crossover were formulated after a pilot run (Section 3.5), whose timings may have influenced them.

Prediction	Statement	Reason	Checked with
P1 Loop order	`ikj` is faster than `ijk` at every $n \geq 500$, and its cost per multiply-add has no jump between $n = 1500$ and 2000	B and C are read row by row, with unit stride	Cost per multiply-add
P2 Blocking	The blocked kernel is faster than `ikj` once B no longer fits in the L3 ($n \geq 2000$)	Each tile of B is reused from the cache by the rows of a tile of A	Ratio to `ikj`
P3 Transposed	Faster than `ijk`, but slower than `ikj`	Unit stride, but every element of C is one serial chain of n additions	Ratio to `ikj`
P4 Vectorisation	The AVX2 build speeds up only the loops gcc reports as vectorised (`ikj`, blocked), by at most 4 times	4 doubles per 256-bit instruction	Ratio AVX2 / scalar
P5 Sparse time	The CSR kernel is faster than the scalar dense blocked kernel up to at least 50% density, and the advantage at a given density is larger for band inputs than for uniform ones	The work is about n^3d^2 multiply-adds, plus costs that grow with the nonzeros of C, which fill in faster for uniform inputs	Ratio of medians
P6 Sparse memory	CSR needs less memory than the dense arrays only while the product stays sparse; for uniform inputs with $d \geq 5\%$ it needs more	12 bytes per stored nonzero against 8 per dense entry	Computed storage, peak working set

Table 1: Predictions and how each one is evaluated.

2 Background

2.1 Dense optimisations

The triple loop performs n^3 multiply-adds whatever the loop order [1]; the order only changes which memory is read in the innermost loop. In the `ijk` order the innermost index k walks down a column of B, $8n$ bytes between consecutive accesses, so every access may touch a different cache line and page. Interchanging the two inner loops (`ikj`) keeps $A[i][k]$ in a register and walks along a row of B and a row of C, so consecutive accesses are adjacent and every 64-byte line serves 8 consecutive multiply-adds [2]. `ikj` still reads the whole of B once for every row of C. *Cache blocking* splits the three loops into $b \times b$ tiles, so a tile of B is reused by all the rows of a tile of A while it is still in the cache; b must be small enough for the tiles to fit, and the best value depends on the cache sizes and on the interference between tiles [4]. An alternative to reordering is to transpose B once, in $O(n^2)$ operations, so that the `ijk` dot product reads both operands with unit stride.

Floating-point addition is not associative, so a change in the order of the additions of one element changes its rounding [3]. All the kernels of this study add the n products of each $C[i][j]$ in increasing k, starting from zero, so they only reorder the visits to different

elements and must give results identical bit for bit to `ijk`. This also means that the dot product of the transposed kernel remains one serial chain of additions: each addition has to wait for the previous one.

Vectorisation executes one instruction on several values: an AVX2 instruction works on 256-bit registers, 4 doubles. gcc vectorises loops automatically with `-ftree-vectorize`, which `-O2` enables with the `very-cheap` cost model; `-fvect-cost-model=dynamic` also allows loops whose vector version needs a run-time check, for example that two pointers do not overlap [7], and `-mavx2` allows the AVX2 instructions [8]. Vectorising the innermost loop of `ikj` over j computes 4 different elements of C at once and does not change the order of the additions of any of them.

2.2 Sparse matrices and the CSR format

The Compressed Sparse Row (CSR) format stores the nonzeros of an $n \times n$ matrix row by row in two arrays, their values and their column indices, plus an array of $n + 1$ offsets where each row starts [6]. With 8-byte values, 4-byte column indices and 8-byte offsets, a matrix with z nonzeros needs $12z + 8(n + 1)$ bytes, against $8n^2$ for the dense array, so CSR is smaller only while fewer than about two thirds of the entries are nonzero.

Gustavson's algorithm computes $C = AB$ row by row: row i of C is the sum of the rows k of B scaled by the nonzeros $A[i][k]$ [5]. It accumulates the row in a dense array of n values and keeps the list of the columns already touched. For inputs with density d and independent uniformly placed nonzeros, it performs about $n \cdot (dn) \cdot (dn) = n^3 d^2$ multiply-adds, a fraction d^2 of the dense work, but each one involves an indirect access through a column index. The product fills in: an element $C[i][j]$ is nonzero when at least one k has $A[i][k] \neq 0$ and $B[k][j] \neq 0$, which for uniform inputs happens with probability $1 - (1 - d^2)^n$. At $n = 2000$ this is 18% for $d = 1\%$ and 99% for $d = 5\%$, so the product becomes dense long before the inputs do. In a band matrix the nonzeros are confined to $|i - j| \leq w$; the product of two such matrices is confined to $|i - j| \leq 2w$, so its fill-in is bounded by about twice the input density.

3 Methodology

3.1 Reused material

The baseline is the validated C implementation of Assignment 1 (repository https://github.com/Cafabri/Assigment1, commit `0785887`). Its kernel is used unchanged, with only the function renamed, and so are its random generator and seeds, timer, batching rule, processor affinity and peak-memory measurement. C was chosen because it was the fastest implementation of Assignment 1 and its flat row-major array makes the effect of the memory-access order directly observable; Java and Python were not continued, so that the comparisons of this study do not mix a language effect with the optimisation and sparse-storage effects.

3.2 Implementations

All the kernels are written in C, single-threaded, and call no numerical library; SciPy is used only to check results.

- **`ijk`** (baseline): the triple loop of Assignment 1.
- **`ikj`** (loop reordering): the j and k loops interchanged; $A[i][k]$ is read once per row of B.
- **`blocked`** (cache blocking): `ikj` on $b \times b$ tiles, with the tile loops in the order ii, kk, jj; the kk loop runs outside the jj loop and in increasing order, so every element still receives its products in increasing k. Partial tiles at the edges are handled with bounds.
- **`transposed`**: the `ijk` loop on B^T, prepared before the call; the transposition itself (`transpose`) is timed as a separate method.
- **`csr`**: Gustavson's algorithm on CSR inputs (own code, sparse.c). A symbolic pass counts the nonzeros of every row of C so that C is allocated once with its exact size; a numeric pass accumulates each row in a dense array of n doubles, with an array of n markers recording which columns the row already holds. The column indices of each row are then sorted: with a scan of the markers when the row has more than $n/16$ nonzeros, otherwise with `qsort`.
- **`convert`**: conversion of the dense A and B to CSR (two passes: count, then copy), timed separately from the kernel.

The CSR product adds the products of every $C[i][j]$ in increasing k, like the dense kernels; the dense kernels also add the products with a zero factor, which are exactly $+0$ and leave the sum unchanged, so its nonzero values are also bit-identical to the `ijk` result.

3.3 Input data

The values are those of Assignment 1: the SplitMix64 generator with seed 42 for A and 43 for B, uniform in $[0, 10)$, in row-major order. SplitMix64 is a counter-based generator, so the value of position $p = i \cdot n + j$ can be computed directly, and a position has the same value whatever the density. Three distributions decide which positions are nonzero:

- **dense**: every position, which gives exactly the matrices of Assignment 1;
- **uniform**: every position independently, when a second SplitMix64 stream (seeds 1042 and 1043) gives a number in $[0, 1)$ below d;
- **band**: only positions with $|i - j| \leq w$, where w is the smallest half-width whose band holds at least dn^2 positions; each band position is kept with probability dn^2/(positions of the band), so the expected density is d (`results/patterns.csv` lists w and the probability of every input).

Because the mask uses the same random number at every density, the nonzeros of a sparser matrix are a subset of those of a denser matrix with the same distribution. The sparse method generates its inputs directly in CSR, so that large sparse inputs never exist as dense arrays; the dense methods and the conversion use the dense arrays of the same

logical matrices. The program checks that the CSR generator and the conversion of the dense array give identical arrays.

3.4 Environment

Item	Value
CPU	Intel Core i7-13700KF: 8 P-cores with 2 threads each (logical processors 0–15) and 8 E-cores (16–23); 16 cores, 24 logical processors
Cache	48 KB L1 data and 2 MB L2 per P-core, 30 MB shared L3
RAM / OS	31.8 GiB / Windows 11 Pro, power plan *Balanced* (default)
Affinity	Runner and every benchmark process restricted to the P-cores (0–15)
Compiler	gcc 16.1.0 (MSYS2/MinGW64)
Build `scalar`	`-O2 -ffp-contract=off` (the flags of Assignment 1)
Build `avx2`	`-O2 -ffp-contract=off -fvect-cost-model=dynamic -mavx2`
Analysis	CPython 3.11.9, NumPy 2.2 and SciPy 1.15 (reference results), pandas 2.2, matplotlib 3.10

Table 2: Execution environment, identical for all runs and recorded by the runner in `results/environment.txt`; the L1 size was read with `Win32_CacheMemory`.

The `scalar` build keeps the flags of Assignment 1: `-O2` without `-O3`, which could interchange the loops by itself [7], and `-ffp-contract=off`, which forbids fused multiply-add instructions that would round differently. The `avx2` build adds only the two vectorisation options. The runner compiles both builds with `-fopt-info-vec`, and the compiler report (`results/vectorisation_report.txt`) is the evidence of which loops are vectorised. In the scalar build gcc vectorises no loop of the kernels. In the AVX2 build it vectorises the innermost loops of `ikj` and of the blocked kernel with 32-byte vectors (4 doubles), each behind a run-time check that B and C do not overlap, and the counting loop of the dense $\rightarrow$ CSR conversion. The dot products of `ijk` and `transposed` are not vectorised: they are floating-point sums, which gcc only vectorises when it is allowed to reorder the additions (`-fassociative-math`) [7]. The `transposed` kernel therefore serves as a control: the same source in both builds, with no vector code in either. The same source code is used in both builds.

3.5 Measurement protocol

A pilot run with a few sizes and densities, made with the final programs, was used to choose the sizes and densities and to test the pipeline; its results are not reported. The script `run_experiment.py` compiles both builds and runs every configuration in a separate process, never two at the same time, in four experiments:

- **Block size:** the blocked kernel for $b \in \{16, 32, 64, 128, 256\}$ at $n \in \{1000, 2000\}$ in both builds. The block size used afterwards, b^*, is the one with the lowest median at $n = 2000$ in each build (rule declared before running).
- **Dense (Part A):** `ijk`, `ikj`, blocked (b^*), `transposed` and `transpose` in the scalar build, and `ikj`, blocked and `transposed` in the AVX2 build, for $n \in \{100, 200, 500, 1000, 1500, 2000, 3000, 4000\}$ on the dense matrices.

- **Density (Part B):** at $n = 2000$, for $d \in \{0.1, 0.5, 1, 2, 5, 10, 20, 30, 50, 75, 100\}\%$ and both distributions: `csr` and `convert` in the scalar build (no loop of the CSR kernel is vectorised in either build), blocked in both builds, and `ijk` only at $d \in \{0.1, 10, 100\}\%$, to check that the dense time does not depend on the zeros.
- **Size (Part B):** `csr` for $n \in \{1000, 2000, 4000, 8000, 16000, 32000\}$ at $d \in \{0.1, 1\}\%$ and both distributions, and the scalar blocked kernel on the same inputs up to $n = 4000$.
- **Budget:** 15 minutes per run, as in Assignment 1, and 16 GiB of matrix storage per process, half of the RAM. A run over the time limit is killed and recorded as TIMEOUT. The program computes the storage it needs before allocating (for CSR, after the symbolic pass) and stops with MEMORY_LIMIT above the budget. In both cases the larger sizes of the same series are recorded as SKIPPED, with no time assigned.
- **Timer:** `QueryPerformanceCounter` around the kernel call only. Dense kernels receive their inputs and a zeroed result prepared outside the timer, as in Assignment 1. The CSR timer includes the symbolic pass, the allocation and sorting of the output and, in batches, freeing the outputs of all but the last call: unlike a dense result, the size of a CSR result is unknown before the multiplication. Its inputs are already in CSR; the conversion is reported separately and, where stated, added to the kernel time. The two boundaries are never combined without saying so.
- **Short runs:** the first call of every process is timed; if it lasts less than 10 ms it is a separate *check* call, and the runs are timed in batches whose size is doubled until a batch lasts at least 10 ms, reporting the time per call. The checksum of these configurations comes from the check call.
- **Warm-up and repetitions:** 2 warm-up runs (the first call is the first warm-up when it is not a check call) and 5 measured runs per process. Configurations whose first process had a median below 60 s were repeated in 3 independent processes interleaved across configurations (15 runs). Each configuration is summarised by the median and the IQR of its measured runs.
- **Result consumption:** every run writes the checksum of its result, or the batch keeps the last result for the checksum of the check call, so no call can be removed by the compiler.

3.6 Memory measurement

Two metrics are reported. The *peak working set* of every timing process, read with `GetProcessMemoryInfo` after the process has exited, is measured and includes the program, the inputs, the outputs and the work space. The *storage* of the matrices of each method is computed from their sizes: $3 \times 8n^2$ bytes for the dense kernels, $4 \times 8n^2$ with B^T; for `csr`, $12z + 8(n + 1)$ bytes per matrix (A, B and C) plus $12n$ bytes of work space (accumulator and markers). Dense and sparse processes hold only the arrays of their own method, so their peak working sets are comparable.

3.7 Correctness

Validation was done in four steps:

1. **Exact cases:** at start-up, every kernel, including the CSR one, checks the product by the identity and zero matrices, [[1,2],[3,4]] × [[5,6],[7,8]] = [[19,22],[43,50]], and a 7×7 sparse product with an empty row and partial 3×3 tiles against `ijk`.
2. **Reference implementation:** for $n = 50$ and 257 (a prime size, so every block size leaves partial tiles) and five inputs (dense; uniform and band at 5% and 30%), the full result of every kernel, every block size and both builds is compared element by element with NumPy, and the nonzeros of the CSR product with SciPy's. The inputs are regenerated independently in NumPy. All values are non-negative, so the relative rounding error of a sum of n terms is bounded by about nu, with $u = 2^{-53}$ [3], 4.4×10^{-13} at $n = 4000$; the tolerance is a relative error of 10^{-11}, and an element whose reference is zero must be exactly zero, because all its products are zero.
3. **Bitwise agreement:** every result of the previous step must be identical bit for bit to the scalar `ijk` result.
4. **Checksums and nonzeros:** for every configuration, the checksum must be the same in every run and agree with the NumPy/SciPy sum within the tolerance, and the nonzeros of A, B and C must equal SciPy's. For the largest sparse products (more than 10^8 nonzeros) the reference is not computed and only the agreement between runs is checked.

4 Results

4.1 Correctness

All validation steps passed (Table 3). The 17 kernels checked (every method, every block size and both builds) produced results identical bit for bit to the scalar `ijk` result, as expected since they add the same products in the same order, and the checksum was identical in every recorded run of every configuration. The baseline reproduces Assignment 1: `ijk` took 0.467, 10.9 and 73.8 s at $n = 1000$, 2000 and 3000, against 0.47, 10.4 and 73.2 s there.

4.2 Part A: block size

Table 4 and Figure 1 give the time of the blocked kernel for each block size. In the scalar build the differences are small and not monotonic: $b = 32$ is the fastest at both sizes, and the slowest block size ($b = 64$) needs 1.17–1.19 times its time. In the AVX2 build the time falls as b grows: $b = 16$ needs 1.81–1.85 times the time of $b = 256$, and $b = 128$ and 256 differ by at most 2.4%. Following the declared rule, the scalar build uses $b^* = 32$ and the AVX2 build $b^* = 256$ in the rest of the study.

Test	Sizes	Largest deviation	Result
Identity, zero, 2×2 and 7×7 cases	2–7	exact	passed (all)
Element-wise vs NumPy (170 comparisons)	50, 257	rel. error 1.9×10^{-15}	passed (all)
All kernels = scalar `ijk`, element-wise	50, 257	bit-identical	passed
Nonzeros of the CSR product vs SciPy	50, 257	exact	passed (all)
Checksums vs NumPy/SciPy (186 configurations)	all tested	rel. difference 2.6×10^{-13}	passed (all)
Checksums across runs	all tested	bit-identical	passed

Table 3: Correctness checks (tolerance: relative error $\leq 10^{-11}$, zeros exact). The two largest uniform products ($n = 16\,000$ and $32\,000$ at 1%) have no reference checksum. Details in `results/correctness.csv` and `results/checksums.csv`.

	Scalar build		AVX2 build	
b	**n = 1000**	**n = 2000**	**n = 1000**	**n = 2000**
16	0.252 (0.9%)	2.02 (0.5%)	0.147 (1.3%)	1.19 (1.3%)
32	0.222 (0.3%)	1.77 (0.7%)	0.101 (2.1%)	0.810 (0.7%)
64	0.261 (3.1%)	2.11 (1.4%)	0.0876 (2.6%)	0.707 (1.0%)
128	0.247 (1.2%)	1.98 (0.8%)	0.0815 (2.3%)	0.661 (1.6%)
256	0.236 (0.7%)	1.92 (1.1%)	0.0815 (2.3%)	0.646 (0.8%)

Table 4: Median time of the blocked kernel in seconds, with the IQR as a percentage of the median in parentheses (15 runs in 3 processes each).

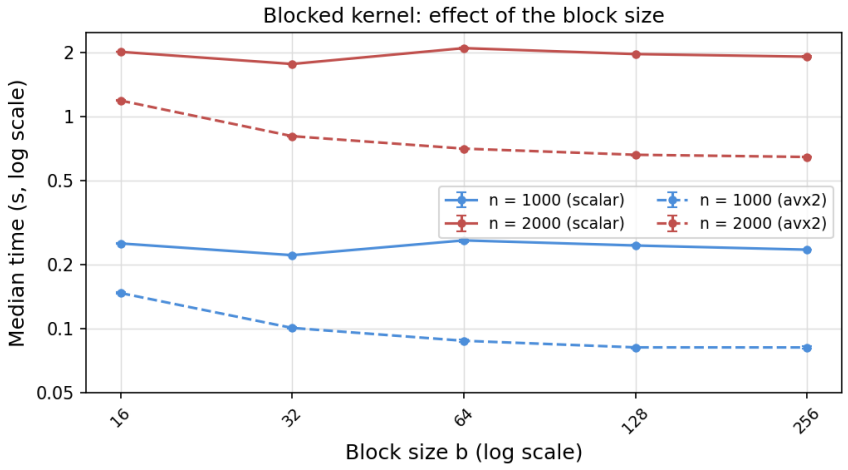


Figure 1: Median time of the blocked kernel vs block size b (log-log scale). Solid: scalar build; dashed: AVX2 build.

4.3 Part A: runtime of the dense methods

Tables 5 and 6 and Figures 2 and 3 give the runtime of every dense configuration; every derived number quoted below is computed by `analysis/report_numbers.py`.

n	ijk (s)	ikj (s)	blocked (s)	transposed (s)	transpose (s)
100	3.01×10^{-4} (5.2%)	2.48×10^{-4} (1.9%)	2.26×10^{-4} (2.3%)	2.57×10^{-4} (4.5%)	3.85×10^{-6} (7.9%)
200	2.72×10^{-3} (1.2%)	1.80×10^{-3} (3.0%)	1.76×10^{-3} (2.3%)	2.84×10^{-3} (3.0%)	1.07×10^{-5} (3.5%)
500	0.0468 (1.6%)	0.0285 (1.0%)	0.0274 (2.1%)	0.0459 (2.0%)	2.13×10^{-4} (9.0%)
1000	0.467 (1.3%)	0.232 (1.0%)	0.219 (2.1%)	0.378 (1.6%)	9.00×10^{-4} (3.3%)
1500	1.77 (9.4%)	0.858 (1.4%)	0.741 (1.0%)	1.35 (2.2%)	2.40×10^{-3} (20.6%)
2000	10.9 (4.7%)	2.84 (2.7%)	1.77 (1.1%)	3.76 (3.4%)	6.40×10^{-3} (33.4%)
3000	73.8 (0.2%)	10.7 (1.4%)	5.97 (0.8%)	13.3 (0.5%)	0.0208 (1.3%)
4000	234 (1.2%)	27.0 (2.8%)	14.2 (1.6%)	32.4 (1.7%)	0.0477 (9.8%)

Table 5: Scalar build: median time per multiplication in seconds, with the IQR as a percentage of the median in parentheses. *transpose* is the transposition of B alone, not included in *transposed*. Every value comes from 15 measured runs in 3 processes, except `ijk` at $n = 3000$ and 4000 (5 runs in one process). For $n \le 200$ each run is a batch of calls lasting at least 10 ms.

	ns per multiply-add							Speedup over scalar `ijk`					
	scalar				AVX2			scalar			AVX2		
n	ijk	ikj	blk	tr	ikj	blk	tr	ikj	blk	tr	ikj	blk	tr
100	0.301	0.248	0.226	0.257	0.088	0.0887	0.305	1.2	1.3	1.2	3.4	3.4	0.99
200	0.34	0.225	0.22	0.355	0.0878	0.0866	0.341	1.5	1.5	0.96	3.9	3.9	1
500	0.374	0.228	0.219	0.367	0.0816	0.0785	0.36	1.6	1.7	1	4.6	4.8	1
1000	0.467	0.232	0.219	0.378	0.0974	0.0805	0.37	2	2.1	1.2	4.8	5.8	1.3
1500	0.524	0.254	0.22	0.399	0.116	0.0803	0.389	2.1	2.4	1.3	4.5	6.5	1.3
2000	1.36	0.355	0.221	0.47	0.224	0.0802	0.456	3.8	6.2	2.9	6.1	17	3
3000	2.73	0.397	0.221	0.493	0.271	0.0803	0.493	6.9	12	5.5	10	34	5.5
4000	3.65	0.423	0.222	0.506	0.296	0.0798	0.498	8.6	16	7.2	12	46	7.3

Table 6: Median time divided by n^3 (nanoseconds per multiply-add) and median time of scalar `ijk` divided by the median time of each method at the same n (blk: blocked; tr: transposed, without the transposition).

Loop order and blocking. `ikj` is faster than `ijk` at every size, by a factor that grows from 1.6 at $n = 500$ and 2.0–2.1 at $n = 1000$–1500 to 3.8 at $n = 2000$ and 8.6 at $n = 4000$. Its cost per multiply-add is 0.23 ns up to $n = 1000$ and then rises to 0.25, 0.36 and 0.42 ns at $n = 1500$, 2000 and 4000; `ijk` rises from 0.52 to 1.36 ns between $n = 1500$ and 2000 and reaches 3.65 ns at $n = 4000$. The blocked kernel keeps 0.219–0.222 ns from $n = 500$ to 4000 (fitted exponent 3.01): it is 1.02–1.16 times faster than `ikj` up to $n = 1500$ and 1.61, 1.79 and 1.90 times faster at $n = 2000$, 3000 and 4000, and 16.4 times faster than `ijk` at $n = 4000$.

Transposed operand. The `transposed` kernel takes 0.96–1.02 times the time of `ijk` at $n = 200$ and 500 (0.36–0.37 ns per multiply-add in both) and is faster from $n = 1000$ (1.23 times) to $n = 4000$ (7.2 times). It is slower than `ikj` at every size, by 1.6 times for $200 \le n \le 1500$ and 1.2 times at $n = 4000$. The transposition itself takes 0.15–0.5% of the time of the `transposed` kernel for $n \ge 200$, so adding it changes the speedup over `ijk` by at most 0.01.

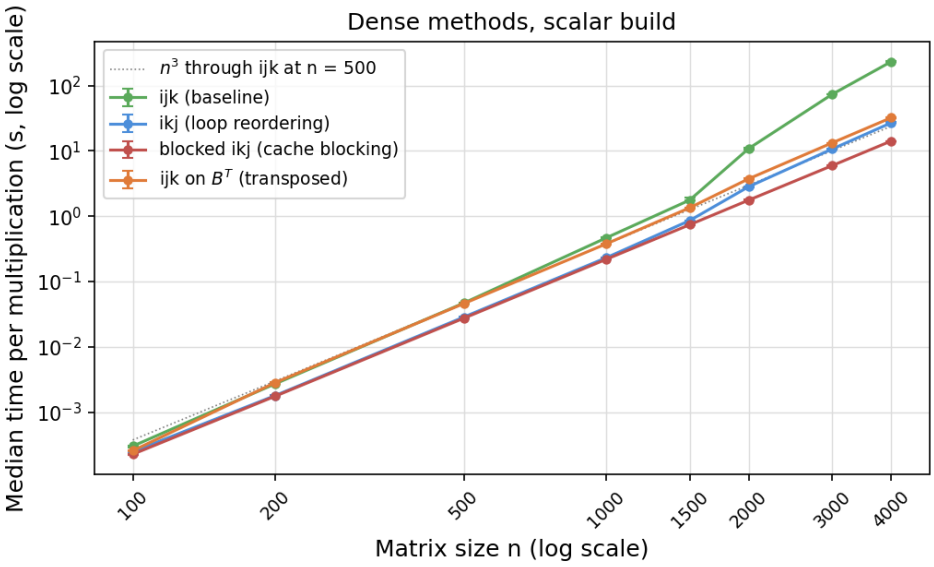


Figure 2: Median time per multiplication of the dense methods in the scalar build vs matrix size n (log-log scale). The dotted line is the cubic model through `ijk` at $n = 500$. Error bars show the IQR (smaller than the markers).

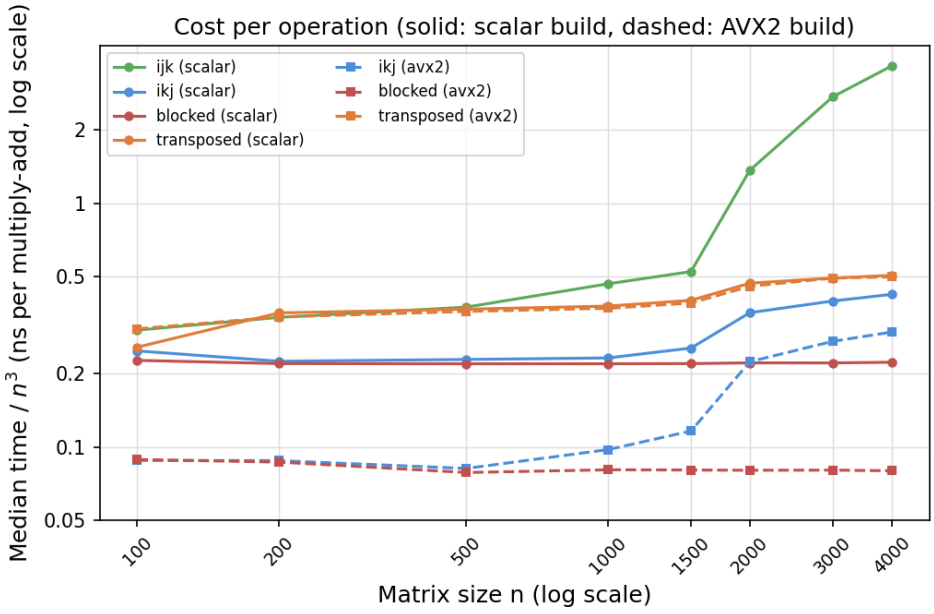


Figure 3: Median time divided by n^3 (nanoseconds per multiply-add), both builds. If the runtime were exactly proportional to n^3, each line would be horizontal.

Vectorisation. The AVX2 build makes the blocked kernel 2.54–2.79 times faster at every size (0.080 ns per multiply-add from $n = 500$ to 4000). For `ikj` the gain falls with n: 2.2–2.8 times up to $n = 1500$, 1.59 at $n = 2000$ and 1.43 at $n = 4000$, where the vectorised `ikj` needs 3.71 times the time of the vectorised blocked kernel. The `transposed` kernel, which has no vector code, takes 0.96–1.00 times its scalar time from $n = 200$; at $n = 100$ the AVX2 build is 16% slower (0.305 against 0.257 ns). The fastest configuration, the vectorised blocked kernel, is 3.4–6.5 times faster than `ijk` up to $n = 1500$ and 17.0, 34.0 and 45.8 times faster at $n = 2000$, 3000 and 4000.

Variability. The IQR is at most 3% of the median in 78.5% of the 214 configurations of the study. Among the configurations with a median above 1 ms, the largest values are those of the transposition alone (up to 33.4% at $n = 2000$, where a call lasts 6 ms), `ijk` at $n = 1500$ (9.4%) and in the density experiment at $n = 2000$ (up to 13.5%, with process medians up to 22% apart), and the vectorised `ikj` at $n = 1500$ (10.8%). Every configuration ran in 3 processes except `ijk` at $n = 3000$ and 4000 (one process, medians of 74 and 234 s).

4.4 Part A: memory

Table 7 gives the peak working set of the dense methods. It is identical for `ijk`, `ikj` and the blocked kernel, and for $n \geq 1000$ grows by 24.0 bytes per matrix element: the three float64 matrices, plus about 4 MiB of fixed program memory. `transposed` grows by 32.0 bytes per element, because it also holds B^T. None of the optimisations needs more memory than the baseline except the transposed operand, which needs one more matrix.

	Peak working set (MiB)				
n	**ijk**	**ikj**	**blocked**	**transposed**	**$24n^2$ bytes (MiB)**
100	4.4	4.4	4.4	4.4	0.2
200	5.1	5.1	5.1	5.3	0.9
500	9.8	9.8	9.8	11.7	5.7
1000	27.0	27.0	27.0	34.6	22.9
1500	55.6	55.6	55.6	72.8	51.5
2000	95.6	95.6	95.6	126.2	91.6
3000	210.1	210.1	210.1	278.8	206.0
4000	370.3	370.3	370.3	492.4	366.2

Table 7: Peak working set of the dense methods in the scalar build (median over processes, MiB = 2^{20} bytes) and computed storage of three float64 matrices. The computed storage of `transposed` is $32n^2$ bytes.

4.5 Part B: density at fixed size

Table 8 and Figures 4–6 give the results at $n = 2000$. The densities of the generated inputs are within 0.02 percentage points of the requested ones.

Dense methods. The dense kernels do not depend on the zeros: across the 22 inputs the median of the blocked kernel varies by 1.6% in each build (1.758–1.787 s scalar, 0.638–0.648 s AVX2), and `ijk` took 9.8–11.5 s, against 10.9 s on the dense matrices.

Fill-in. For uniform inputs the measured density of C follows the model $1 - (1 - d^2)^n$ (Figure 5): 18% at $d = 1\%$, 55% at $d = 2\%$ and 99.3% at $d = 5\%$, so from $d = 5\%$ the product is practically dense. For band inputs the density of C is 1.5–2.1 times that of the inputs up to $d = 20\%$ (37.8%), and the product reaches 100% only at $d = 75\%$.

Time. The CSR kernel is faster than the scalar blocked kernel up to $d = 50\%$ in both distributions: by 10 100 (uniform) and 18 900 (band) times at $d = 0.1\%$, 187 and 1000 times at $d = 1\%$, 16.9 and 30.7 times at $d = 10\%$, and 1.23 and 1.37 times at $d = 50\%$. At $d = 75\%$ and 100% it needs 1.9 and 3.4 times the dense time. At every density from 0.1% to 50% the advantage is larger for band inputs. Interpolating the ratio in logarithmic scale, the time crossover is at 55.1% (uniform) and 57.3% (band) against the scalar blocked

	Density (%)				Dense blocked (s)		Scalar blocked /	
d (%)	A	C	CSR (s)	Conv. (s)	scalar	AVX2	CSR	CSR + conv.
Uniform nonzeros								
0.1	0.1	0.2	1.73×10^{-4}	7.84×10^{-3}	1.76	0.640	10100	219
0.5	0.49	4.81	5.31×10^{-3}	8.76×10^{-3}	1.77	0.646	333	126
1	0.99	18	9.52×10^{-3}	8.98×10^{-3}	1.78	0.645	187	96
2	1.99	55	0.0295	1.00×10^{-2}	1.77	0.642	59.9	44.7
5	4.99	99.3	0.0641	0.0139	1.76	0.643	27.5	22.6
10	9.98	100	0.106	0.0207	1.79	0.647	16.9	14.1
20	20	100	0.268	0.0333	1.76	0.643	6.58	5.85
30	30	100	0.536	0.0420	1.76	0.641	3.29	3.05
50	50	100	1.43	0.0529	1.76	0.639	1.23	1.18
75	75	100	3.38	0.0379	1.76	0.648	0.522	0.516
100	100	100	6.04	0.0175	1.76	0.642	0.292	0.291
Band nonzeros								
0.1	0.1	0.15	9.32×10^{-5}	7.90×10^{-3}	1.76	0.642	18900	220
0.5	0.5	1.03	7.66×10^{-4}	7.97×10^{-3}	1.77	0.642	2310	202
1	1	2.03	1.76×10^{-3}	8.03×10^{-3}	1.76	0.638	1000	180
2	2	4.01	5.11×10^{-3}	8.37×10^{-3}	1.76	0.641	346	131
5	5	9.98	0.0172	8.57×10^{-3}	1.77	0.641	102	68.4
10	10	19.6	0.0572	9.26×10^{-3}	1.76	0.640	30.7	26.4
20	20	37.8	0.211	9.89×10^{-3}	1.76	0.645	8.36	7.98
30	30	54.7	0.465	0.0109	1.77	0.644	3.8	3.72
50	50	82.9	1.30	0.0128	1.77	0.645	1.37	1.35
75	75	100	3.27	0.0153	1.76	0.643	0.54	0.537
100	100	100	6.06	0.0175	1.77	0.640	0.291	0.291

Table 8: $n = 2000$. Requested density d, measured density of A and of the product C, median times of the CSR kernel, of the conversion of A and B to CSR and of the dense blocked kernel, and ratio of the scalar blocked time to the CSR time without and with the conversion. A ratio above 1 means that CSR is faster.

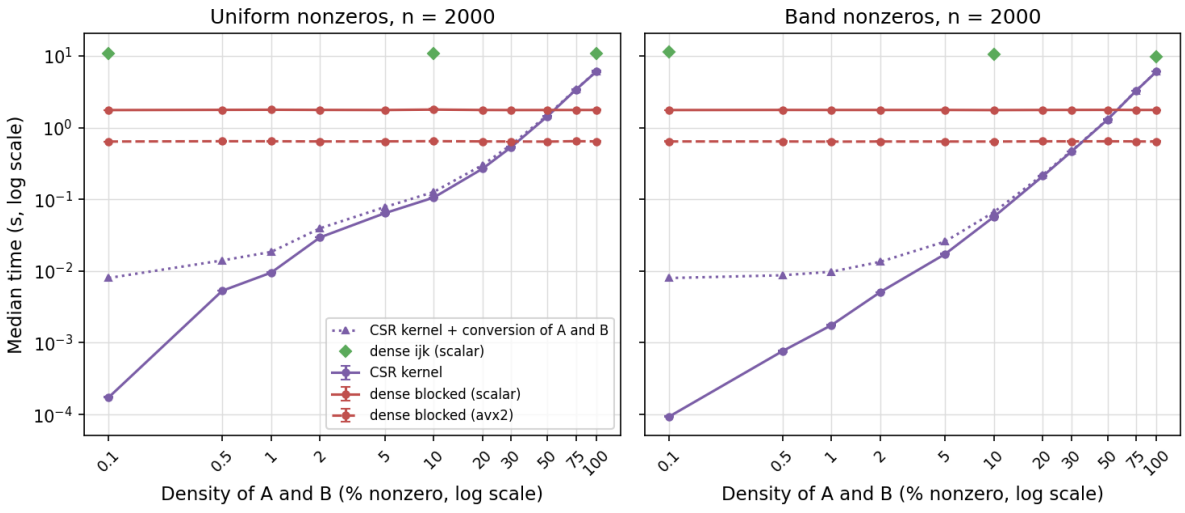


Figure 4: Median time vs density of A and B at $n = 2000$ (log-log scale). Left: uniform nonzeros; right: band nonzeros. The dense `ijk` was run at three densities only.

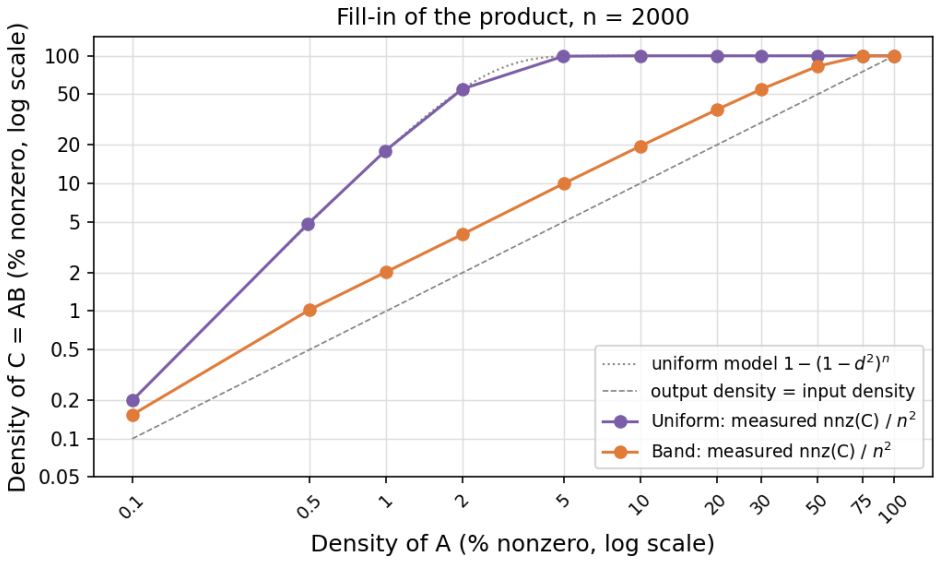


Figure 5: Density of the product $C = AB$ vs density of the inputs at $n = 2000$ (log-log scale). The dotted line is the uniform model $1 - (1 - d^2)^n$.

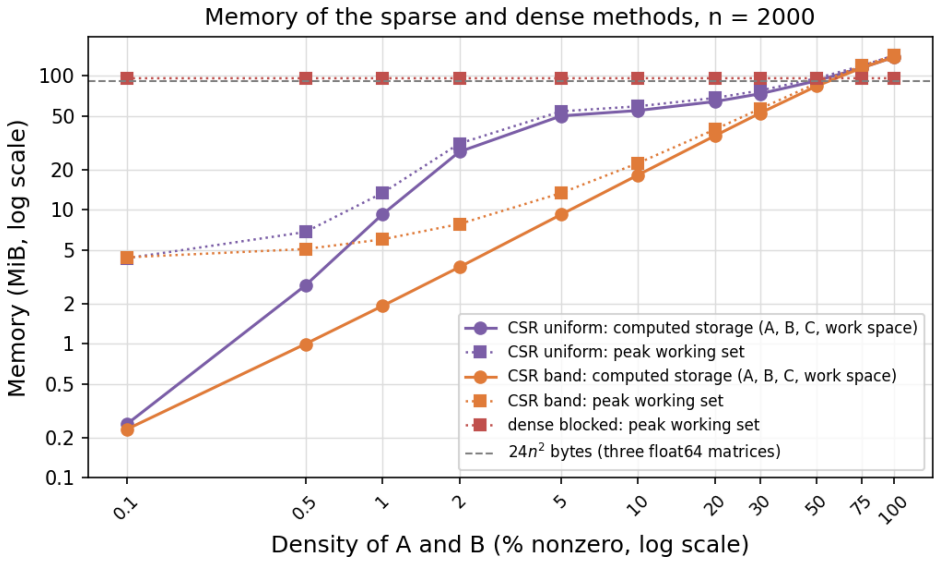


Figure 6: Memory vs density at $n = 2000$ (log-log scale): computed storage of the CSR matrices A, B, C and work space, and peak working set of the CSR and dense blocked processes. The dashed line is $24n^2$ bytes.

kernel, and at 32.9% and 35.3% against the vectorised one. Divided by n^3d^2, the CSR time is 0.72–0.76 ns (uniform) and 0.65–0.76 ns (band) for $d \geq 20\%$, and grows at low density, up to 21.7 and 11.6 ns at $d = 0.1\%$.

Conversion. Converting A and B to CSR takes 7.8–53 ms. It costs 45 (uniform) and 85 (band) times the CSR kernel at $d = 0.1\%$, about as much as the kernel at 0.5–1% (uniform) and 2–5% (band), and less than 20% of it from $d = 10\%$. Including it moves the crossovers by at most 1.2 percentage points (54.3% and 57.1% against the scalar kernel) and leaves CSR 219 and 220 times faster than the scalar dense kernel at $d = 0.1\%$. For

uniform inputs the conversion time is highest at $d = 50\%$ (52.9 ms) and falls to 17.5 ms at $d = 100\%$; for band inputs it grows steadily with d.

Memory. The computed storage of the CSR matrices is smaller than the $24n^2$ bytes of the three dense matrices up to 49.9% density (uniform) and 56.1% (band), and 1.5 times larger at 100%. At $d = 5\%$, with an almost full product, the uniform CSR matrices still need 0.55 times the dense storage. The peak working set follows the same pattern: 4.3 MiB at $d = 0.1\%$ and 141.5 MiB at $d = 100\%$ for CSR, against 95.6 MiB for the dense blocked kernel at every density.

4.6 Part B: matrix size at fixed density

Table 9 and Figure 7 give the CSR results from $n = 1000$ to 32 000. The fitted exponents of the CSR time over this range are 3.33 (uniform 0.1%), 2.74 (uniform 1%), 2.46 (band 0.1%) and 2.52 (band 1%). At a fixed density the uniform products fill in as n grows (from 9.4% to 95.9% at $d = 1\%$), while the band products keep a constant density (0.2% and 2.0%). On the inputs where both ran ($n \leq 4000$), CSR is 158–24 300 times faster than the scalar blocked kernel.

Distr.	**d (%)**	**n**	**CSR (s)**	**C (%)**	**Storage**	**Peak**	**Blocked (s)**
Uniform	0.1	1000	1.32×10^{-5} (4.7%)	0.098	0	4	0.219
		2000	1.73×10^{-4} (3.5%)	0.2	0	4	1.77
		4000	1.61×10^{-3} (1.4%)	0.4	1	5	14.1
		8000	0.0159 (1.2%)	0.8	8	12	—
		16000	0.154 (0.7%)	1.59	53	57	—
		32000	1.45 (0.4%)	3.15	394	398	—
	1	1000	1.39×10^{-3} (1.9%)	9.37	1	5	0.220
		2000	9.40×10^{-3} (1.5%)	18	9	13	1.77
		4000	0.0662 (1.6%)	32.9	64	68	14.1
		8000	0.488 (0.5%)	55	418	422	—
		16000	3.08 (0.3%)	79.8	2397	2401	—
		32000	17.3 (0.4%)	95.9	11477	11481	—
Band	0.1	1000	9.72×10^{-6} (3.8%)	0.1	0	4	0.219
		2000	9.38×10^{-5} (8.3%)	0.15	0	4	1.76
		4000	5.79×10^{-4} (1.6%)	0.2	1	5	14.1
		8000	2.58×10^{-3} (2.6%)	0.21	3	7	—
		16000	0.0110 (2.3%)	0.2	12	17	—
		32000	0.0617 (1.7%)	0.2	48	53	—
	1	1000	3.83×10^{-4} (1.9%)	2.05	0	5	0.220
		2000	1.77×10^{-3} (2.5%)	2.03	2	6	1.77
		4000	0.0102 (2.3%)	2.01	7	12	14.1
		8000	0.0561 (0.9%)	2	30	34	—
		16000	0.345 (1.0%)	2	118	122	—
		32000	2.35 (0.6%)	1.99	469	473	—

Table 9: CSR multiplication at fixed density: median time (IQR in parentheses), density of the product, computed storage and peak working set (MiB), and the scalar dense blocked kernel on the same inputs (run up to $n = 4000$).

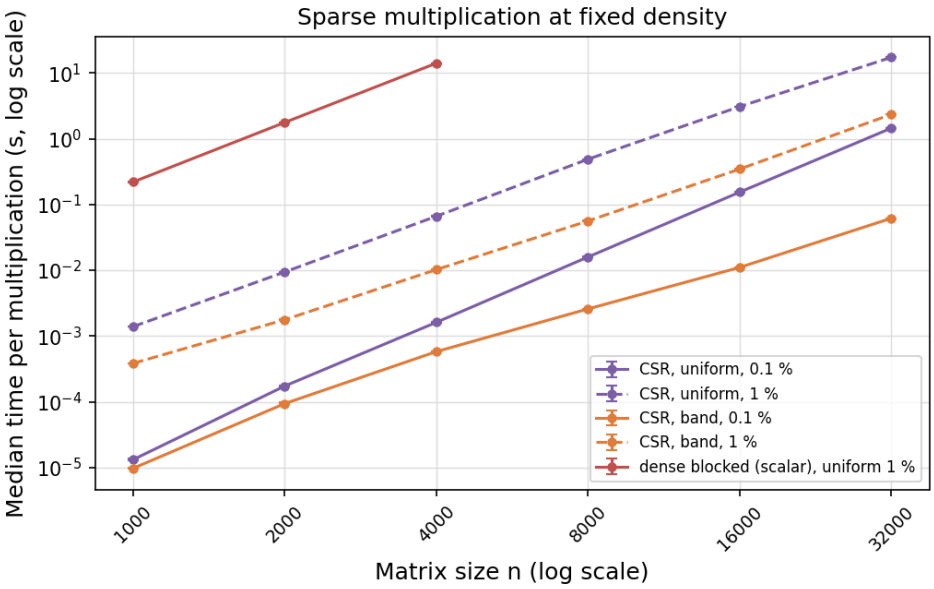


Figure 7: Median time of the CSR multiplication at fixed density vs matrix size n (log-log scale), and the dense blocked kernel (scalar) up to $n = 4000$.

4.7 Largest feasible size

No process exceeded the time or memory budget, so every declared size finished (`results/run_status.csv`). Table 10 summarises the limits.

Method	**Largest feasible n**	**Longest median**	**Largest peak**
`ijk` (scalar)	4000	234 s	370 MiB
`ikj` / blocked (scalar)	4000	27.0 s / 14.2 s	370 MiB
blocked (AVX2)	4000	5.1 s	370 MiB
`transposed` (scalar)	4000	32.4 s	492 MiB
CSR, uniform 1%	32 000	17.3 s	11 481 MiB
CSR, band 1%	32 000	2.35 s	473 MiB

Table 10: Limits under the budget of 15 minutes per run and 16 GiB of matrix storage. The largest sizes attempted were $n = 4000$ for the dense methods and 32 000 for CSR; none exceeded the budget.

5 Discussion

5.1 Evaluation of the hypotheses

In answer to question (A): reading memory with unit stride and reusing tiles from the cache remove most of the slowdown of `ijk` at large n without extra memory, and vectorisation multiplies the gain of the kernels that gcc can vectorise. In answer to question (B): at $n = 2000$, CSR is faster than the dense kernels up to a density of about one half against the scalar kernel and one third against the vectorised one, and needs less storage up to about one half, for both distributions. Table 11 gives the verdict for each prediction.

Prediction	Observed	Verdict
P1 Loop order	`ikj` faster at every $n \geq 500$ (1.6–8.6 times), but its cost per multiply-add still rises from 0.25 to 0.36 ns between $n = 1500$ and 2000	Partly confirmed (smaller jump, not none)
P2 Blocking	1.61–1.90 times faster than `ikj` for $n \geq 2000$; constant 0.22 ns per multiply-add	Confirmed
P3 Transposed	Slower than `ikj` at every size; faster than `ijk` only from $n = 1000$, equal within 4% for $200 \leq n \leq 500$	Partly confirmed (not faster below $n = 1000$)
P4 Vectorisation	Blocked 2.5–2.8 times, `ikj` 1.4–2.8 times, the non-vectorised `transposed` within 4% from $n = 200$	Confirmed
P5 Sparse time	CSR faster than scalar blocked up to 50% (crossovers 55% and 57%); larger advantage for band inputs at every density up to 50%	Confirmed
P6 Sparse memory	CSR storage smaller than dense up to 50% (uniform) and 56% (band), including uniform inputs with $5\% \leq d \leq 50\%$	Refuted (threshold wrong)

Table 11: Verdict for each prediction of Table 1.

5.2 Effect of the dense optimisations

Memory access. The four dense kernels perform the same n^3 multiply-adds in the same order for each element, so their differences come from the order in which memory is read. For $n \leq 1500$, B (at most 18 MB) fits in the 30 MB L3, and `ikj` is 1.6–2.1 times faster than `ijk`. Beyond that size the column walk of `ijk` becomes 2.6 times more expensive per operation, while `ikj` rises by 40%. `ikj` reads B row by row, so every page and cache line it touches is used completely; its smaller jump is therefore consistent with the L3-capacity explanation of Assignment 1 (the whole of B, 32 MB at $n = 2000$, is read again from memory for every row of C), and suggests that the much larger jump of `ijk` comes from its stride in addition. Neither the TLB nor the cache misses were measured, so the share of each effect remains a hypothesis. At $n = 2000$, `ikj` reads one 8-byte element of B per multiply-add, which at 0.36 ns (scalar) and 0.22 ns (AVX2) is equivalent to about 22 and 36 GB/s, plausible main-memory rates; no bandwidth benchmark was run to confirm that these are the limit.

The blocked kernel removes the dependence on n: its cost per multiply-add is the same from $n = 500$ to 4000, as expected if each tile of B is reused from the cache by the b rows of a tile of A instead of being read again from memory [4]. The best block sizes are consistent with the cache sizes, but this was not verified directly: with $b = 32$ the three tiles occupy $3 \times 8b^2 = 24$ KB, within the 48 KB L1, and with $b = 256$ they occupy 1.5 MB, within the 2 MB L2. In the scalar build all the block sizes are within 19%, so the exact choice matters little; in the AVX2 build, small tiles are 1.8 times slower, consistent with the inner loop over b elements being too short for the vector loop (4 doubles per instruction, unrolled 4 times) and the run-time aliasing check that precedes it.

The transposed operand. Transposing B gives `ijk` unit stride but keeps its dot product, one serial chain of n additions per element. Up to $n = 500$, where `ijk` still finds B in the cache, both kernels cost 0.36–0.37 ns per multiply-add, while `ikj`, whose consecutive additions go to different elements, costs 0.23 ns. This is consistent with the dot-product kernels being limited by the latency of the chain of additions rather than by memory [2]; it was not measured with hardware counters. The transposition itself costs almost nothing (at most 0.5% of the kernel time), so the transposed operand is a valid fix for the memory problem of `ijk` but not as good as reordering the loops.

Vectorisation. Separating the build from the method shows that the order of the loops is what allows vectorisation: the inner loop of `ikj` updates independent elements of C and is vectorised, whereas the dot product cannot be vectorised without changing the rounding, and `transposed` gains nothing. The gain never reaches the 4 doubles of an AVX2 register (at most 2.8), and in `ikj` it falls to 1.4 at large n: once the arithmetic is 2.5–2.8 times faster, the kernel that reads B from memory for every row spends a larger share of its time waiting for memory, which the blocked kernel avoids. The two optimisations therefore combine: blocking alone gives 16.4 times over `ijk` at $n = 4000$, vectorisation alone (`ikj`) 12.4 times with the loop reordering it needs, and both together 45.8 times. The 16% slowdown of the non-vectorised `transposed` at $n = 100$ is not explained by the measurements; the code is the same in both builds, so a difference in code alignment is one possible cause.

5.3 Effect of sparse storage

The CSR kernel avoids all the products with a zero factor, about $n^3(1 - d^2)$ of the n^3 multiply-adds, but pays more for each product it performs. For $d \geq 20\%$ its time divided by n^3d^2 is 0.65–0.76 ns, 3.0–3.5 times the 0.22 ns of the scalar blocked kernel and 8–9.5 times the 0.080 ns of the vectorised one: each product reads a column index, updates the accumulator at a scattered position and checks a marker. If both costs were constant, the crossover would be where d^2 equals their ratio, at $\sqrt{0.22/0.72} \approx 55\%$ against the scalar kernel and $\sqrt{0.080/0.72} \approx 33\%$ against the vectorised one, close to the measured 55% and 33% (uniform). The crossover thus depends on how optimised the dense kernel is: the vectorisation of the dense kernel moves it from one half to one third.

At low density the cost per product grows (up to 21.7 ns at $d = 0.1\%$), because the number of products falls as d^2 while some costs do not: the n rows of C, the initialisation of the markers and the sorting of each row of C. These costs grow with the nonzeros of C, which is why the distribution matters: at $d = 1\%$ both distributions perform about the same number of products, but the uniform product has 18% nonzeros and the band product 2%, and the band kernel is 5.4 times faster. The band rows of B that a row of A selects are also adjacent in memory, which may help; this was not separated from the effect of the fill-in.

The conversion of the inputs is negligible at medium and high densities but dominates at very low density (7.8 ms against 0.17 ms for the kernel at $d = 0.1\%$, uniform): its time is bounded below by reading the $2 \times 8n^2$ bytes of the two dense matrices twice. Even so, CSR with conversion remains about 220 times faster than the scalar dense kernel at $d = 0.1\%$. The peak of the uniform conversion time at $d = 50\%$ is consistent with the branch `if (v != 0)` of the copy pass being unpredictable when half of the random entries are zero, which is not the case for band inputs, whose nonzeros form almost contiguous runs (each

band position is kept with probability above 0.95 from $d = 1\%$); branch mispredictions were not measured.

Memory. Prediction P6 compared the 12 bytes of a CSR nonzero with the 8 bytes of a dense entry for the product only. The comparison that matters is with all three dense matrices: even when C is full and its CSR form needs $12n^2$ bytes, 1.5 times its dense form, the sparse A and B save more than that up to about $d = 50\%$. Above it, CSR needs up to 1.5 times the dense storage, and its peak working set at $d = 100\%$ is 141.5 MiB against 95.6 MiB.

Scaling. At a fixed density the number of products grows as n^3d^2, but the measured exponents are not 3. For band inputs (2.5) the fixed per-row costs dominate at small n and are amortised as n grows: the time per product of band 1% falls from 3.8 ns at $n = 1000$ to 0.71 ns at 32 000. For uniform inputs at 0.1% the exponent is above 3: the rows of C have about d^2n^2 nonzeros (about 1000 at $n = 32\,000$), which are sorted with `qsort`, so the cost per product grows with $\log n$; this is consistent with, but was not confirmed by, a profile of the kernel. The scaling experiment also shows the main limit of sparse storage: uniform inputs with 1% density give a product with 96% nonzeros at $n = 32\,000$, and the CSR storage (11.2 GiB, mostly C) is larger than the 7.6 GiB of a dense C. For comparison, three dense matrices would need $24n^2$ bytes, 22.9 GiB, above the 16 GiB budget, and the cubic extrapolation of the blocked kernel gives about 2 hours per run; neither was run. Band inputs, whose product stays sparse, needed 473 MiB at the same size.

5.4 Limitations

- One Windows computer, one compiler and one set of flags; other cache sizes would move the jumps of the dense kernels, the best block size and the crossovers. The power plan was the default *Balanced*, not a fixed frequency.
- The block size was chosen among five powers of two at one size; rectangular tiles, register blocking and the cache-oblivious recursion were not tested, so the blocked kernel is not the fastest possible dense kernel. The densities tested are 11 values, and the crossovers between them are interpolated.
- The cache, TLB, latency and branch-prediction explanations are inferred from timings only; no hardware counters or profiles were used.
- Part B was measured at $n = 2000$ only, and with two nonzero distributions; the crossovers depend on n through the fill-in, and other patterns (block, power-law rows) may behave differently.
- The CSR implementation was not compared with an optimised library, and its sorting rule ($n/16$) was not tuned.
- `ijk` at $n = 3000$ and 4000 ran in one process (5 runs); `ijk` at $n = 2000$ showed process medians up to 22% apart, so its single-process values may underestimate the uncertainty.
- No configuration reached the budget, so the actual largest feasible sizes were not determined.

6 Conclusions

For the programs and the conditions described in Section 3:

- Changing only the order of memory accesses makes the dense multiplication much faster at the same number of operations and with the same memory: `ikj` is 1.6–8.6 times faster than `ijk` for $500 \leq n \leq 4000$ and the blocked kernel 1.7–16.4 times. The blocked kernel keeps a constant cost of 0.22 ns per multiply-add up to $n = 4000$, and the jump of Assignment 1 does not appear in it.
- Transposing B also removes most of the slowdown of `ijk` at large n, at a negligible cost and one more matrix of memory, but remains 1.2–1.6 times slower than `ikj`.
- Compiler vectorisation multiplies the effect of the optimisations it can be applied to: 2.5–2.8 times for the blocked kernel, up to 45.8 times faster than `ijk` at $n = 4000$, and nothing for the dot-product kernel.
- At $n = 2000$, CSR is faster than the dense blocked kernel up to a density of about 55–57% (scalar) and 33–35% (vectorised), by four orders of magnitude at 0.1%, and needs less storage up to 50–56%. Its advantage at a given density is larger when the product stays sparse (band) than when it fills in (uniform).
- The conversion to CSR is negligible except at very low densities, where it costs up to 85 times the sparse kernel without changing the conclusion.

These results describe specific implementations on one machine. The effect of the optimisations is to make each operation cheaper; the effect of sparse storage is to avoid operations at a higher cost per operation, so its advantage depends both on the density and on how fast the dense alternative is. Two questions remain open: which share of the slowdown of the dense kernels comes from the cache and which from address translation, which hardware counters could answer, and how the crossovers move with n and with the use of several threads.

Reproducibility materials

The repository linked on the first page contains the source code, the build and run commands (`README.md`) and, in `results`, the raw measurements of every run, the status and peak memory of every process, the patterns of the sparse inputs, the block size chosen, the compiler vectorisation report, the environment record and the correctness results. The study is reproduced on Windows with `run_experiment.py` (build and benchmarks, 3.3 hours of benchmark processes on the machine described above), the validation script `analysis/check_correctness.py`, `analysis/plot_results.py` (figures and tables) and `analysis/report_numbers.py` (every other number in the text).

Use of generative AI

Claude Opus 5.5 (Anthropic), used through Claude Code, was used during this assignment to help write and debug the benchmark, run and analysis code, and to draft and revise the text of the report from the measured results. All the measurements come from running the code on the machine described above. The author defined the study, reviewed the code and the AI-generated text, checked the numbers against the raw data and the references, and is responsible for the conclusions.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[3] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163.

[4] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74, doi: 10.1145/106972.106981.

[5] F. G. Gustavson, "Two fast algorithms for sparse matrices: multiplication and permuted transposition," *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978, doi: 10.1145/355791.355796.

[6] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[7] Free Software Foundation, "Options that control optimization," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html.

[8] Free Software Foundation, "x86 options," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html.